Frotend

React

App

UnityClient

ApiClient

Components

«Component» My

«Component» DataForm

«Component» Media

«Component» CurrentShapeCard

«Component» Navbar

«Component» Login

«Component» LoginForm

«Component» UserStats

«Component» Dashboard

«Component» NotFound

«Component» UserData

Modelos de Datos

Activity
+ uuid: Integer
+ date: Date
+ minigame: String
+ duration: Float
+ score: Float
+ activity_points: Float
+ extra_data: String
+ user_email: String

User
+ uuid: Integer
+ email: String
+ hashed_password: String
+ verified: Boolean
+ name: String
+ year_of_birth: Integer

1 — Realiza — 0..*

1 — Registra — 0..*

Shape
+ uuid: Integer
+ date: Date
+ weight: Float
+ height: Float
+ user_email: String

Backend

Cabe destacar que los routers **no** son realmente clases.
Realmente se trata de funciones agrupadas en módulos que se llaman de manera automática al recibir una petición HTTP.

API Routers

«Router» ActivityRouter
+ create_activity(activity: ActivitySchema, current_user: UserSchema) -> ActivitySchema
+ get_activities(current_user: UserSchema, minigame_filter: str | None = None, from_date: str | None = None, to_date: str | None = None) -> list[ActivitySchema]
+ delete_activity(uuid: int, current_user: UserSchema) -> None

«Router» ShapeRouter
+ create_shape(shape: ShapeSchema, current_user: UserSchema) -> ShapeSchema
+ get_shapes(current_user: UserSchema, from_date: str | None = None, to_date: str | None = None) -> list[ShapeSchema]
+ get_current_shape(current_user: UserSchema) -> ShapeSchema | None
+ delete_shape(uuid: int, current_user: UserSchema) -> None

«Router» UserRouter
+ create_user(user: UserSchema) -> UserSchema
+ send_verification_email(current_user: UserSchema) -> None
+ verify_user(token: str) -> None
+ get_me(current_user: UserSchema) -> UserSchema
+ update_me(modifications: UserSchema, current_user: UserSchema) -> UserSchema
+ update_email(token: str) -> None
+ delete_me(current_user: UserSchema) -> None

«Router» AuthRouter
+ login_for_token(form_data: dict) -> Token
+ check_token(token: str) -> str

creates, receives, returns

creates, receives, returns

depends on

creates, returns

creates, receives, returns

creates, returns

creates, receives, returns

returns

depends on

depends on

depends on

Schemas

Los esquemas son clases que definen la estructura de los datos que se envían y reciben a través de la API. Se utilizan para validar y serializar los datos.

ActivitySchema
+ date: Date
+ minigame: String
+ duration: Float
+ score: Float
+ activity_points: Float
+ extra_data: String

ShapeSchema
+ date: Date
+ weight: Float
+ height: Float

UserSchema
+ email: String
+ name: String
+ year_of_birth: Integer

TokenSchema
+ access_token: String
+ token_type: String

creates, manages

creates, returns

manages

creates, manages

creates, manages

DAO

ActivityDAO
+ create_activity(activity: ActivitySchema, user_email: str) -> ActivitySchema
+ get_activities(user_email: str, minigame_filter: str | None, from_date: datetime, to_date: datetime) -> list[ActivitySchema]
+ delete_activity(uuid: int, user_email: str) -> None

ShapeDAO
+ create_shape(shape: ShapeSchema, user_email: str) -> ShapeSchema
+ get_shapes(user_email: str, from_date: datetime, to_date: datetime) -> list[ShapeSchema]
+ get_current_shape(user_email: str) -> ShapeSchema | None
+ delete_shape(uuid: int, user_email: str) -> None

UserDAO
+ create_user(user: UserSchema) -> UserSchema
+ update_user(current_user: UserSchema, modification: UserSchema) -> UserSchema
+ get_user(email: str) -> UserSchema | None
+ verify_user(token: str) -> bool
+ delete_user(current_user: UserSchema) -> None

AuthDAO
+ get_user(uuid: int) -> UserSchema | None
+ get_user_by_email(email: str) -> UserSchema | None